So sánh MCU – SoC – FPGA

Đánh đổi giữa hiệu năng, chi phí và độ linh hoạt

CO3053 Hệ thống nhúng · HK261 · Đề tài 09

01/11/2026

Tóm tắt. Không nền tảng nào thắng trên mọi tiêu chí. MCU thắng về giá, năng lượng khi nghỉ và chứng nhận an toàn. SoC thắng khi sản phẩm cần hệ điều hành, mạng và đa phương tiện. FPGA thắng khi cần thông lượng song song cố định hoặc độ trễ cỡ nano-giây, đổi lại là giá cao và khó lập trình.
Nhóm tự kiểm chứng bằng cùng một bộ lọc FIR 16-tap Q15 trên cả ba nền tảng, và mọi nền tảng phải khớp từng mẫu với một golden model chung. Trong bài, mỗi con số đều mang một nhãn cho biết nguồn gốc: [đo], [mô phỏng], [ước lượng công cụ], [tính] hoặc [nguồn].

1 Câu hỏi và phạm vi

Bài trả lời một câu hỏi: *khi nào nên chọn MCU, SoC hay FPGA*, xét theo hiệu năng, chi phí và độ linh hoạt. Bài không nhằm chứng minh một nền tảng là "tốt nhất", vì đặt mục tiêu thiên vị là một lỗi thường gặp mà `pe2` đã cảnh báo.

Nền tảng	Đại diện	Cách chạy phần mềm
MCU	ARM Cortex-M: STM32F103 (nhóm đo), STM32F4, STM32H743, STM32U5 (tài liệu)	Bare-metal hoặc RTOS
SoC	ARM Cortex-A + Linux: Raspberry Pi 4 (nhóm đo), STM32MP1	Linux có sẵn driver, mạng, file system
FPGA	Artix-7 XC7A100T trên Arty A7-100T (nhóm đo); Zynq làm phương án lai	Mô tả phần cứng bằng HDL, không có chương trình tuần tự

Bảng 1: Ba nền tảng được so sánh. Zynq (CPU và FPGA trên cùng chip) được xếp vào *phương án lai*, không gọi là SoC.

Giá linh kiện của MCU và FPGA là giá chip, còn giá của Raspberry Pi là giá cả board. Vì vậy các phép so sánh giá chỉ dùng để thấy bậc độ lớn.

2 Kiến trúc: vì sao ba nền tảng khác nhau

- MCU: một lõi CPU nhỏ (Cortex-M), có flash và SRAM ngay trên chip, nhiều ngoại vi như timer, ADC, PWM. Chip chạy lệnh tuần tự và *không có MMU*, nên không chạy được Linux. Đổi lại, MCU khởi động trong vài mili-giây và đáp ứng ngắt trong vài trăm nano-giây.
- SoC: một hoặc nhiều lõi ứng dụng (Cortex-A) có MMU và cache, cùng GPU, bộ giải mã video và các khối tăng tốc mạng trên một chip. Chip chạy được Linux, nên có sẵn driver và network stack. Cái giá là độ trễ khó đoán trước, vì hệ điều hành lập lịch, cache và ngắt đều xen vào.

- **FPGA**: một mảng LUT, flip-flop, khối DSP và BRAM có thể nối lại tùy ý. Thuật toán được "trải ra không gian": mỗi phép nhân có một khối nhân riêng chạy song song, nên thông lượng có thể đạt 1 mẫu/chu kỳ và độ trễ cố định theo số chu kỳ. Cái giá là mọi chức năng đều phải tự mô tả ở mức phần cứng.

3 Số liệu từ tài liệu

3.1 Hiệu năng

	Chip	Lõi / xung nhịp	CoreMark	
MCU	STM32F4	Cortex-M4, 180 MHz	608	[nguồn] [1]
MCU	STM32H743	Cortex-M7, 480 MHz	2 424	[nguồn] [2]
SoC	STM32MP1	Cortex-A7, 800 MHz	4 158	[nguồn] [1]
SoC	Raspberry Pi 4	4×Cortex-A72, 1,8 GHz	38 200 (4 lõi)	[nguồn] [3]

Bảng 2: CoreMark (càng cao càng nhanh). Chỉ số này không áp dụng cho FPGA, vì FPGA không chạy chương trình tuần tự.

Thông lượng MAC. Bài toán lọc ảnh 3×3 trên video 1080p, 3 kênh màu, 30 khung/giây cần khoảng 1,68 G MAC/s (Lee & Seshia, mục 8.1).

- STM32H743 đạt tối đa khoảng 0,48 G MAC/s [tính: 1 MAC/chu kỳ × 480 MHz], tức chỉ khoảng 29% nhu cầu.
- XC7A100T với 240 khối DSP48E1 chạy ở 200 MHz đạt khoảng 48 G MAC/s [tính]; hãng công bố tối đa 264 GMAC/s [4].
- Theo Arucu & Iliev [5], FIR 27-tap trên Zynq XC7Z020 chạy nhanh hơn Cortex-A9 khoảng 27 lần, và năng lượng mỗi lần chạy thấp hơn khoảng 95–190 lần [tính, vì công suất CPU là ước tính của tác giả].

Nền tảng	Độ trễ xấu nhất điển hình	
FPGA	13,9 ns (tick-to-trade, STAC-T0)	[nguồn] [6]
MCU bare-metal	1–3 µs	[nguồn] [7]
SoC Linux PREEMPT_RT, đã tinh chỉnh	30–80 µs	[nguồn] [7]
SoC Linux thường	vài ms	[nguồn] [7]

Bảng 3: Độ trễ xấu nhất theo tài liệu. Các phép đo khác bản chất nhau, chỉ dùng để so bậc độ lớn.

3.2 Chi phí và công suất

4 Thí nghiệm của nhóm

4.1 Phương pháp

Ba nền tảng chạy cùng một bài toán: bộ lọc FIR 16-tap low-pass (Hamming, $f_c = 0{,}2 f_s$, tổng hệ số $= 2^{15}$) ở định dạng Q15. Phép tính được đặc tả như sau:

$$y[n] = \mathrm{sat}_{16}\Big(\big\lfloor \textstyle\sum_{k=0}^{15} h[k]\,x[n-k] \,/\, 2^{15} \big\rfloor\Big).$$

	MCU STM32H743	SoC Raspberry Pi 4	FPGA XC7A100T
Giá mua lẻ	£9,26–11,97 / chip [8]	$35 / board khi ra mắt [9]	€143,90 / chip [10]
Khi chạy	≈0,44 W [tính: 275 µA/MHz × 480 MHz × 3,3 V]	≈7 W đỉnh, cả board [11]	1,431 W đo trên Zynq XC7Z020, gồm cả ARM [5]
Khi nghỉ	2,43 µA ở chế độ Standby [2]; STM32U5: 300 nA [12]	≈3 W khi rảnh, cả board	chưa có trong nguồn đã dùng

Bảng 4: Chi phí và công suất. STM32U5 ở chế độ Standby dùng khoảng 1 µW, ít hơn Pi 4 lúc rảnh khoảng 3 triệu lần [tính, so chip với board].

- **E0, golden model.** Viết bằng Python, sinh 1024 mẫu gồm xung đơn vị, bậc thang, giá trị biên, mẫu đối kháng ép bão hòa (60 mẫu ra bị bão hòa) và mẫu ngẫu nhiên. Mọi nền tảng phải khớp *từng mẫu* với kết quả này.
- **E1, MCU STM32F103.** Cortex-M3 chạy ở *chưa đo* MHz.
  - Bốn phiên bản FIR: C dịch đường trễ, C bộ đệm vòng khai triển, CMSIS `arm_fir_q15` và `arm_fir_fast_q15`.
  - Đếm chu kỳ bằng `DWT->CYCCNT` khi đã tắt ngắt.
  - Đo độ trễ ngắt: TIM2 ở 10 kHz, ISR đọc `TIM2->CNT` ngay lệnh đầu tiên, lấy 20 000 mẫu cho mỗi kịch bản.
- **E2, FPGA Arty A7-100T.**
  - RTL pipeline 7 tầng, có bão hòa. Testbench chạy hai lượt: liên tục và có khoảng trống ngẫu nhiên trên `in_valid`.
  - Trên board: đọc 1024 mẫu từ BRAM ở 100 MHz, so với golden ngay trên chip, đếm chu kỳ và báo kết quả qua UART.
  - Vivado cho Fmax, số tài nguyên và công suất [ước lượng công cụ].
- **E3, SoC Raspberry Pi 4.**
  - Cùng mã C với MCU, ghim vào một lõi, build `-O2` (không NEON) và `-O3` (GCC tự vector hóa); lấy median của 15 lần chạy.
  - Đo độ trễ lập lịch bằng `cyclictest` (SCHED_FIFO 95) khi rảnh và khi có tải `stress-ng`.

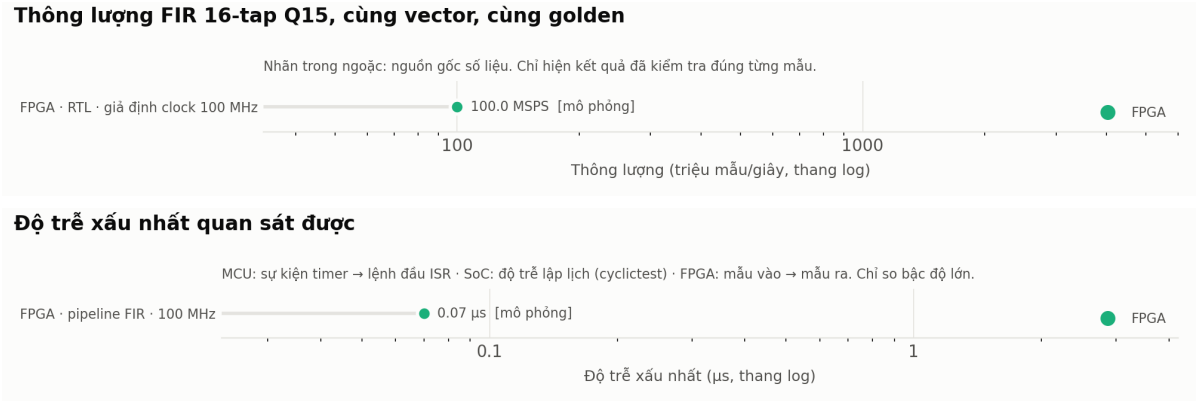
4.2 Kết quả

4.3 Kiểm chứng chéo với tài liệu

- **Giả thiết 1 MAC/chu kỳ.** Con số ≈0,48 G MAC/s của H743 ở trên giả định 1 MAC/chu kỳ. Trên M3, bản tốt nhất của nhóm đạt *chưa đo*% mức đó. Nếu tỉ lệ này thấp, con số lý thuyết chỉ là cận trên lạc quan. Lý do: ngoài phép nhân, CPU còn phải nạp hai toán hạng, cập nhật đường trễ, và flash có 2 wait state ở 72 MHz.
- **FPGA so với CPU.** FPGA ở 100 MHz nhanh hơn Pi 4 *chưa đo* lần và nhanh hơn MCU *chưa đo* lần với bài 16-tap này. Arucu & Iliev đo được khoảng 27 lần so với Cortex-A9 với bài 27-tap. Khi so, cần nhớ khác biệt về chip, số tap và cách đo.
- **Một quan sát phụ.** Khi chạy thử code trên laptop x86, nhóm thấy một lõi CPU desktop lọc được khoảng 140 MSPS với bài 16-tap. Như vậy FPGA không thắng chỉ nhờ tốc độ thô ở bài nhỏ. Lợi thế của FPGA nằm ở chỗ khác:

	Đại lượng	Giá trị	
MCU	Chu kỳ/mẫu, bản tốt nhất (*chưa đo*)	*chưa đo*	[đo]
MCU	Chu kỳ/mẫu, C dịch đường trễ	*chưa đo*	[đo]
MCU	Thông lượng tốt nhất (MSPS)	*chưa đo*	[đo]
MCU	Hiệu suất so với giả thiết 1 MAC/chu kỳ (%)	*chưa đo*	[tính từ số đo]
MCU	Độ trễ ngắt khi chạy FIR, min–max (ns)	*chưa đo*	[đo]
MCU	Như trên, thêm ngắt ưu tiên cao hơn chạy 20 µs (µs, xấu nhất)	*chưa đo*	[đo]
SoC	Thông lượng FIR tốt nhất, 1 lõi A72 (MSPS)	*chưa đo*	[đo]
SoC	`cyclictest` xấu nhất khi có tải, Linux thường (µs)	*chưa đo*	[đo]
SoC	`cyclictest` xấu nhất khi có tải, PREEMPT_RT (µs)	*chưa đo*	[đo]
FPGA	Kiểm tra với golden	PASS	[mô phỏng]
FPGA	Mẫu/chu kỳ; thông lượng ở 100 MHz (MSPS)	1,00; 100,0	[mô phỏng]
FPGA	Độ trễ pipeline	7 chu kỳ = 70 ns	[mô phỏng]
FPGA	Fmax của riêng khối FIR (MHz); số DSP48E1 (trên 240)	*chưa đo*; *chưa đo*	[ước lượng công cụ]
FPGA	Công suất cả chip ở 100 MHz (W)	*chưa đo*	[ước lượng công cụ]

Bảng 5: Kết quả thí nghiệm, tự động lấy từ `experiments/results/` qua `collect.py`. Ô ghi *chưa đo* nghĩa là chưa có phép đo; nhóm không điền số giả định vào những ô này.



Hình 1: Thông lượng và độ trễ xấu nhất trên trục log. Nhãn trong ngoặc vuông cho biết nguồn gốc số liệu.

  - thông lượng giữ nguyên khi tăng số tap, chỉ tốn thêm DSP;
  - độ trễ cố định theo số chu kỳ;
  - công suất thấp.

  Số đo trên laptop chỉ để kiểm tra code, không đưa vào bảng.

- **Độ trễ Linux.** Kết quả `cyclictest` trên Pi 4 được đặt cạnh các con số 30–80 µs và "vài ms" của AC6 [7].
- **Worst-case của MCU.** Kịch bản có ngắt ưu tiên cao hơn cho thấy worst-case của MCU do thiết kế phần mềm quyết định: ở đây là *chưa đo* µs, trong khi khi không có ngắt chen ngang chỉ vài trăm ns. Nói "MCU có độ trễ 1–3 µs" chỉ đúng khi hệ thống được thiết kế cho mục tiêu đó.

5 Ứng dụng: mỗi nền tảng chiếm ưu thế ở đâu và vì sao

Phần này chọn những lĩnh vực mà mỗi nền tảng gần như là lựa chọn mặc định. Mỗi mục trả lời ba câu: *điều gì quyết định, vì sao nền tảng này đáp ứng*, và *vì sao hai nền tảng còn lại không phù hợp*. Mỗi mục cũng nêu chỗ mà ưu thế đó dừng lại.

5.1 MCU: ECU điều khiển trên ô tô

Điều gì quyết định: an toàn chức năng, nhiệt độ khắc nghiệt, thời gian cung ứng dài, giá thấp ở số lượng lớn.

- *Số lượng.* Một xe phổ thông có khoảng 25–35 MCU, xe sang khoảng 60–70 [15]. Riêng mảng powertrain và chassis chiếm khoảng 30% thị trường MCU ô tô [16]. Infineon nắm khoảng 32% thị phần MCU ô tô năm 2024 [17].
- *An toàn chức năng.* Họ AURIX TC3xx có bốn trong sáu lõi chạy *lockstep*: hai lõi chạy cùng một lệnh và so kết quả mỗi chu kỳ. Họ chip này nhắm tới mức ASIL-D của ISO 26262, dùng cho powertrain, chassis và inverter xe điện [18].
- *Môi trường.* Chuẩn AEC-Q100 Grade 0 yêu cầu chip chạy được từ −40 đến +150 °C. Ví dụ, NXP S32K14xW đạt Grade 0 và được cam kết cung ứng ít nhất 15 năm [19].
- *Vì sao MCU đáp ứng:* khởi động trong vài ms, đáp ứng ngắt dưới 1 µs (xem E1), đã có sẵn chứng nhận an toàn, và giá vài đô la một chip.
- *Vì sao không chọn SoC hay FPGA:* Linux mất nhiều giây để khởi động và có độ trễ không xác định. FPGA đắt, cấu hình nằm trong SRAM nên có nguy cơ lỗi do bức xạ (SEU), và khó chứng nhận cho số lượng hàng triệu xe.
- *Ưu thế dừng ở đâu:* ADAS và hệ thống giải trí dùng SoC (xem mục 5.4). Xu hướng *domain controller* đang gom nhiều ECU nhỏ thành ít bộ điều khiển lớn hơn [15].

Các lĩnh vực MCU cũng chiếm ưu thế:

- Điều khiển động cơ BLDC/PMSM: vòng FOC 20 kHz phải xong trong 50 µs, và thư viện của ST chạy dưới 21 µs trên Cortex-M3 [14].
- Nút cảm biến chạy pin: STM32U5 chỉ dùng 300 nA ở chế độ Standby [12], và thiết kế mẫu của TI chạy được hơn 10 năm trên pin CR2032 [13].
- Thiết bị gia dụng và ngoại vi USB như bàn phím, chuột.

5.2 SoC: router Wi-Fi và điện thoại

Điều gì quyết định: cần một hệ điều hành đầy đủ với network stack, *cộng thêm* các khối tăng tốc cố định để đạt hàng Gbit/s mà CPU không phải tự làm.

- *Router Wi-Fi.* SoC router của Qualcomm gom trên một chip:
  - CPU Cortex-A chạy Linux/OpenWrt;
  - radio Wi-Fi;
  - switch Ethernet;
  - khối xử lý gói tin cứng (PPE, 37,5 Mpps trên IPQ6018);
  - khối tăng tốc mã hóa.

  Bản Wi-Fi 7 IPQ9574 có 4 lõi A73 ở 2,2 GHz, switch 6 cổng và khối offload tunnel [20, 21].
- *Điện thoại.* Snapdragon 8 Elite gom CPU Oryon, GPU Adreno, NPU Hexagon, ISP cho camera và modem 5G X80 trên một chip 3 nm [22].

- *Vì sao SoC đáp ứng:* Linux đã có sẵn các chức năng mà sản phẩm cần: routing, firewall, VPN, giao diện web, cập nhật từ xa. Phần nặng như chuyển gói tin hay mã hóa được đẩy sang khối cứng chuyên dụng, nên CPU chỉ lo phần điều khiển (control plane).
- *Vì sao không chọn MCU hay FPGA:* MCU không có MMU nên không chạy được Linux, và phải tự viết lại toàn bộ TCP/IP stack [23]. FPGA làm được phần chuyển gói tin, nhưng viết cả network stack và giao diện bằng HDL thì quá đắt cho thiết bị tiêu dùng.
- *Ưu thế dừng ở đâu:* SoC không phù hợp với điều khiển thời gian thực cứng, vì PREEMPT_RT vẫn ở mức 30–80 µs [7]. Vì vậy trong router và điện thoại vẫn có thêm MCU nhỏ lo quản lý nguồn và cảm biến.

Các lĩnh vực SoC cũng chiếm ưu thế: camera IP và camera thông minh (camera chạy được ngay qua V4L2 trên Linux [24]), TV và set-top box, gateway IoT, hệ thống giải trí trên xe.

5.3 FPGA: xử lý tín hiệu song song có I/O tốc độ cao, và tạo mẫu chip

Một lưu ý về cách gọi. "Tính toán song song" nói chung thì GPU mới là lựa chọn phổ biến, ví dụ huấn luyện AI hay HPC. FPGA thắng ở một ngách hẹp hơn: dữ liệu *chảy liên tục* từ cảm biến hoặc ADC, phải xử lý với *độ trễ cố định*, và chuẩn hoặc thuật toán thay đổi nhanh hay số lượng sản xuất không đủ lớn để làm ASIC.

- *Siêu âm y tế (beamforming).*
  - Đầu dò có 128–256 kênh. Mỗi kênh phải được trễ, nhân trọng số và cộng lại theo thời gian thực.
  - Một FPGA duy nhất đạt 4,83 GSamples/s, và chip Xilinx hiện nay xử lý được tới 1024 kênh [25].
  - FPGA nối thẳng với các chip front-end analog nên tránh được chi phí chuyển dữ liệu qua PCIe như khi dùng GPU [25].
- *Trạm phát sóng 5G (radio unit).*
  - Phần low-PHY và digital front-end (FFT, lọc, beamforming, tiền méo) chạy trên RFSoC của Xilinx hoặc Agilex của Intel. Ví dụ, Fujitsu dùng UltraScale+ RFSoC cho radio unit 5G [26, 27].
  - Chuẩn O-RAN còn đang thay đổi, nên khả năng nạp lại cấu hình giá trị hơn việc đúc một ASIC.
- *Tạo mẫu ASIC và SoC trước khi sản xuất chip.*
  - Hệ thống HAPS của Synopsys ghép nhiều FPGA lại để chạy RTL của chip thật, tới khoảng 1,6 tỉ cổng trên HAPS-80, nhờ đó viết phần mềm được trước khi có chip [28].
  - Ở đây khả năng tái cấu hình chính là mục đích: mỗi lần sửa RTL chỉ cần nạp lại, không phải đúc lại chip.
- *Giao dịch tần suất cao:* tick-to-trade 13,9 ns [6].
- *Vì sao FPGA đáp ứng:* các phép tính độc lập được chạy song song trong phần cứng (bài 16-tap của nhóm dùng *chưa đo* DSP, 1 mẫu/chu kỳ). Độ trễ cố định theo chu kỳ. I/O nối trực tiếp vào chân chip.
- *Vì sao không chọn MCU hay SoC:* MCU thiếu khoảng 3,5 lần năng lực dù đạt hiệu suất 100%. SoC có thể đủ thông lượng nhờ SIMD hoặc GPU, nhưng mất tính xác định và tốn điện hơn.

- *Ưu thế dừng ở đâu:* khi số lượng đủ lớn và thuật toán đã ổn định, ASIC rẻ hơn và tiết kiệm điện hơn; đó là lý do điện thoại dùng ISP và modem cứng chứ không dùng FPGA. Với phép tính dấu phẩy động khối lượng lớn, GPU thường thắng.

5.4 Phương án lai CPU + FPGA: ADAS và điều khiển công nghiệp

Có những bài toán cần *cả* hệ điều hành phong phú *lẫn* timing hoặc xử lý song song cứng.

- Zynq UltraScale+ gom trên một chip: 4 lõi Cortex-A53 chạy Linux, 2 lõi Cortex-R5 real-time chạy lockstep, và fabric FPGA. Dòng chip này nhắm tới ADAS và điều khiển công nghiệp [29].
- Lecture 3 xếp "CPU + FPGA" là tốt cho điều khiển actuator: CPU lo phần tính toán mềm dẻo, FPGA giữ timing nghiêm ngặt kể cả khi CPU đang bận.
- Cái giá là chi phí cao hơn và linh hoạt thấp hơn so với dùng một CPU đơn.

	Lĩnh vực đặc trưng	Yếu tố quyết định	Vì sao không chọn hai nền tảng kia
MCU	ECU ô tô, điều khiển động cơ, cảm biến chạy pin	ASIL-D, 150 °C, dòng nghỉ cỡ nA, giá vài USD	SoC khởi động chậm và độ trễ không xác định; FPGA đắt và khó chứng nhận
SoC	Router Wi-Fi, điện thoại, camera IP	Linux + khối tăng tốc cứng	MCU không có MMU; FPGA tốn công viết network stack bằng HDL
FPGA	Siêu âm, 5G radio, tạo mẫu ASIC, HFT	Song song + I/O trực tiếp + tái cấu hình	MCU thiếu năng lực; SoC mất tính xác định
Lai	ADAS, điều khiển công nghiệp	Cần cả OS lẫn timing cứng	Một nền tảng đơn không đáp ứng đủ cả hai

Bảng 6: Tóm tắt các lĩnh vực đặc trưng của từng nền tảng.

6 Ma trận quyết định

Điều kiện của bài toán	Chọn	Căn cứ
Chạy pin nhiều năm, tác vụ nhẹ	**MCU**	300 nA ở Standby; hơn 10 năm trên CR2032
Deadline vài chục µs, cần độ trễ ổn định, có chứng nhận an toàn	**MCU**	Ngắt dưới 1 µs (E1); AURIX ASIL-D
Cần Linux, camera, mạng, giao diện; timing mềm	**SoC**	CoreMark gấp ≈16 lần H743; driver có sẵn
Trên ≈0,5 G MAC/s liên tục, hoặc I/O tốc độ cao	**FPGA** / lai	≈48 G MAC/s [tính]; 1 mẫu/chu kỳ (E2)
Độ trễ cỡ ns, pipeline cố định	**FPGA**	13,9 ns tick-to-trade
Cần cả OS phong phú lẫn timing cứng	Lai	Zynq; Lecture 3, Option #2

Bảng 7: Ma trận quyết định: điều kiện của bài toán nào dẫn tới nền tảng nào.

7 Hạn chế

- Số liệu tài liệu đến từ nhiều nguồn với điều kiện đo khác nhau. Nhiều nguồn chỉ được đọc qua đoạn trích trong kết quả tìm kiếm, và giá bán lẻ có thể thay đổi theo ngày.
- Thí nghiệm của nhóm chỉ dùng *một* bài toán (FIR 16-tap). Ba nền tảng ở các phân khúc giá khác nhau: F103 khoảng vài USD, Pi 4 khoảng 35 USD, Arty khoảng 300 USD cả board.
- Công suất FPGA là ước lượng của Vivado (vector-less), không phải số đo. Phép đo độ trễ ở ba nền tảng là ba đại lượng khác nhau.
- Phần ứng dụng dựa trên tài liệu của hãng và báo chuyên ngành. Con số thị phần đến từ các công ty nghiên cứu thị trường và có thể lệch nhau.

Tài liệu

[1] ST, *STM32H7/F4/MP1 product overview (CoreMark)*. https://www.st.com/resource/en/product_presentation/microcontrollers_stm32h72x-3x_line_product-overview.pdf

[2] ST, *STM32H743/753*. https://www.st.com/en/microcontrollers-microprocessors/stm32h743-753.html

[3] OpenBenchmarking, *Raspberry Pi 4 CoreMark*. https://mail.openbenchmarking.org/result/2511110-NE-BROADCOMB24&export=html

[4] AMD/Xilinx, *XA Artix-7 datasheet*. https://datasheet.chipsmall.com/XA7A25T-1CPG238Q/297909.pdf

[5] Arucu & Iliev, *J. Low Power Electron. Appl.* 15(3):40, 2025. https://www.mdpi.com/2079-9268/15/3/40

[6] A-Team Insight, *Exegy and AMD STAC-T0*. https://a-teaminsight.com/blog/exegy-and-amd-achieve-record-breaking-tick-to-trade-latency-in-stac-t0-benchmark/

[7] AC6, *Real-time latency of Linux with PREEMPT_RT*. https://www.ac6-formation.com/en/articles/real-time-latency-linux-preempt-rt

[8] RS UK, *STM32H743ZIT6*. https://uk.rs-online.com/web/p/microcontrollers/1653272

[9] AnandTech, *Raspberry Pi 4 launched*. https://at-web1.www.anandtech.com/show/14581/raspberry-pi-4-launched-quad-cortex-a72-project-board-for-35-dollars

[10] Farnell, *XC7A100T-1CSG324C*. https://at.farnell.com/xilinx/xc7a100t-1csg324c/fpga-artix-7-210-i-o-cspbga-324/dp/2760123

[11] TechFeed, *Raspberry Pi 4 power*. https://techfeed.io/entries/5d112019e50460ac6f5ccf16

[12] Arrow, *STM32U5*. https://www.arrow.com/en/research-and-events/articles/stm32u5-ultra-low-power-arm-cortex-m33-32-bit-microcontroller

[13] TI, *TIDU797*. https://ti.com/lit/pdf/tidu797

[14] ST, *Motor control with STM32*, 2008. https://www.promelec.ru/UPLOAD/fck/file/stm_files/stm32/Motor_control_with_STM32.pdf

[15] Embedded World, *Chip crisis highlights importance…*, 02/2026. https://www.embedded-world.de/en/knowledge/2026/02/chip-crisis-highlights-importance-

[16] GM Insights, *Automotive microcontrollers market*. https://www.gminsights.com/industry-analysis/automotive-microcontrollers-market

[17] eeNews Europe, *Top ten automotive microcontroller suppliers in 2024*. https://www.eenewseurope.com/en/top-ten-automotive-microcontroller-suppliers-in-2024

[18] Automotive World, *AURIX TC3xx fuels automated driving and electromobility*. https://www.automotiveworld.com/news/aurix-microcontroller-tc3xx-family-infineon-fuels-automated-driving-electromobility/

[19] Future Electronics, *NXP S32K1 AEC-Q100 microcontrollers*. https://www.futureelectronics.com/npi/nxp-s32k1-aec-q100-microcontrollers

[20] Macnica, *Qualcomm Networking Pro (IPQ6018)*. https://www.macnica.co.jp/en/business/semiconductor/manufacturers/qualcomm/products/134933/

[21] CNX Software, *Qualcomm Wi-Fi 7 Networking Pro Series*, 2022. https://www.cnx-software.com/2022/05/05/qualcomm-wi-fi-7-networking-pro-series-socs-support-up-to-33-gbps-phy-rate-up-to-2000-clients/

[22] GSMArena, *Qualcomm announces Snapdragon 8 Elite*. https://m.gsmarena.com/qualcomm_announces_snapdragon_8_elite_chip_with_oryon_cpu_unreal_engine_53_support_and_40_fater_gpu-news-65009.php

[23] Google Developers, *Computing at the edge of IoT*. https://medium.com/google-developers/computing-at-the-edge-of-iot-140a888007bd

[24] Applied EE, *Where MPUs fit*. https://applied-ee.github.io/ee-notebook/docs/embedded/mpu-and-sbc-architecture/where-mpus-fit

[25] Kou et al., *High-level synthesis design of scalable ultrafast ultrasound beamformer with single FPGA*, 2023. https://pmc.ncbi.nlm.nih.gov/articles/PMC10405367

[26] HPCwire, *Xilinx teams with Fujitsu on 5G O-RAN*, 2021. https://www.hpcwire.com/aiwire/2021/02/04/xilinx-teams-with-fujitsu-on-5g-o-ran/

[27] Intel, *Accelerate Open RAN O-RU deployments with Agilex 7 SoC FPGA*. https://www.intel.com/content/www/us/en/content-details/789219.html

[28] Synopsys, *FPGA-based prototyping (HAPS)*. https://www.synopsys.com/fpga-based-prototyping

[29] eeNews Europe, *Programmable MPSoC targets vision, ADAS, IoT and 5G*. https://www.eenewseurope.com/en/programmable-multi-processor-soc-targets-vision-adas-iot-and-5g/

Tài liệu môn học: Lecture 2 (Choosing a Platform), Lecture 3 (ECU Design Option #1–#3), pe2-PE-steps, pe3-technique-metrics-selection, Lee & Seshia, *Introduction to Embedded Systems*, mục 8.1–8.2. Tra cứu ngày 01/11/2026.